Response Letter

We thank the reviewers for their valuable comments. The paper has been carefully revised according to the meta-review, and all of the requested changes were implemented. Below is our detailed response to the meta-review and each reviewer.

Replies to Requested Changes

Requested Change 1: Correct the description of SEDAR's comment-before-mutation and restore-after-mutation mechanism. Clearly explain the actual methodological differences between MORPH and SEDAR, avoiding misleading characterizations of the latter.
Reply: We apologize for the inaccurate description. The original paper described SEDAR's mutability refinement as "destructive sanitization," which was incorrect: SEDAR temporarily comments out parser-incompatible fragments during mutation and restores them before sending the mutated SQL to the target DBMS, so it does not permanently remove query logic. We have revised the paper as follows:

(1) We have corrected the description of SEDAR throughout the paper. In Section II, we have described its mutability refinement including the restoration step, revised the motivating example to show the commented fragment as an intermediate state that is restored before execution, and changed SEDAR's *Cross-stmt. Deps.* entry in Table I to △. In the Introduction, the old version's sentence "generated test cases are often sanitized by removing or commenting out unsupported constructs, which may simplify complex SQL structures and weaken execution behaviors" is replaced with "parser-incompatible fragments are handled after translation rather than resolved during it. For example, SEDAR temporarily comments them out during mutation and restores them before execution on the target DBMS, which retains the original query logic but leaves translation errors in these fragments unrepaired."

(2) We have clarified the methodological differences in Sections II and VI. SEDAR translates each statement of a test case individually under its referenced source-side sub-schema, whereas MORPH propagates target-side schema and type context across statements via the AQT. Moreover, MORPH repairs incompatibilities with parser and execution feedback before mutation, whereas SEDAR's restoration retains the hidden fragments without repairing translation errors in them. Neither strategy guarantees semantic equivalence of the mutated queries.

(3) We have removed the statements attributing MORPH's improvements to permanent loss of query logic in SEDAR (Sections I, IV-C, IV-D, and VI), and added a controlled comparison in Section IV-C. For each of the four target DBMSs, we applied both mutation-handling strategies to the same 300 translated seeds and ran GRIFFIN for 24 hours with five repetitions. MORPH's handling achieves TODO(a)%, TODO(b)%, TODO(c)%, and TODO(d)% higher branch coverage on DuckDB, MonetDB, Virtuoso, and ClickHouse, respectively.

Requested Change 2: Moderate the novelty claims and better distinguish MORPH from existing SQL translation approaches, particularly SEDAR, RISE, QTRAN, and CrackSQL. Explain more concretely what AQT and its associated mechanisms contribute beyond existing techniques, especially in the context of DBMS fuzzing.
Reply: TODO.

Requested Change 3: Clarify how MORPH differs from and complements SmartFuzz in seed construction. The additional comparison is useful but limited in its experimental scope. The revision should acknowledge these limitations and avoid overstating comparative advantages.
Reply: TODO.

Requested Change 4: Clarify the provenance of the SEDAR binary used in the experiments, particularly given conflicting reports about its public availability. Provide sufficient information about the artifact and configuration to support reproducibility.
Reply: TODO.

Replies to Reviewer A

Comment 1.1: Can the authors evaluate source-to-target result or state equivalence?
Reply: TODO.

Comment 1.2: What independent benefit does the dependency graph provide beyond context propagation?
Reply: TODO.

Comment 1.3: How were CrackSQL, RISE, and SEDAR adapted and configured for the four target DBMSs, and how did the authors ensure a fair comparison?
Reply: TODO.

Comment 1.4: **1. Novelty** SEDAR is the closest predecessor because it addresses the same high-level problem of transferring existing DBMS test cases into fuzzing seeds. SEDAR already processes complete multi-statement test cases. It sequentially executes the source test, captures a referenced source-side sub-schema for each statement, translates individual statements, and restores their original order. Its mutability refinement also temporarily hides regions unsupported by the fuzzer parser during mutation and restores them before sending the resulting SQL to the target DBMS.

MORPH should therefore not be positioned as the first whole-test-case transfer framework or the first system to adapt transferred seeds to downstream fuzzers. Nevertheless, MORPH provides clear technical advances over SEDAR. In particular, it replaces independent statement-level translation with an explicit AQT representation, forward read-after-write dependencies, evolving target-side context, dialect-evidence-guided instantiation, and parser/execution-feedback repair. SEDAR does not maintain translated target context across statements and explicitly leaves target-side execution-feedback regeneration as future work. I consider these differences meaningful.

CrackSQL, RISE, and QTRAN provide relevant methodological precedents but address different tasks. CrackSQL already performs tree-based query decomposition, abstraction of translation-irrelevant content, target-syntax retrieval, localized translation, and operation-scope expansion. RISE reduces a complex query or procedural artifact, extracts a source-to-target AST rule, and applies the rule back to the original artifact. QTRAN uses dialect knowledge and RAG to translate an original query and regenerate its target-side mutant while preserving a metamorphic relation. These works show that structural decomposition, localized translation, and dialect-aware

retrieval are not individually new. However, none of them models an independently executed stateful fuzzing test sequence, maintains target-side context across its statements, or integrates the result with coverage-guided seed mutation in the manner proposed by MORPH.

The related-work discussion contains several details that should be corrected. In particular, the paper states that CrackSQL employs genetic algorithms to mine mutation patterns, but the cited work instead uses functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. In addition, describing RISE as strictly processing a single SQL statement is too narrow because it also evaluates complete stored procedures containing control flow and multiple DML statements. A more accurate distinction is that RISE processes one self-contained query or procedural artifact at a time rather than a test file containing an ordered sequence of independently executed statements.

Overall, the individual translation techniques have precedents, but their extension to explicit stateful seed transfer through AQT, target-context propagation, and feedback-driven repair constitutes a credible contribution.
Reply: TODO.

Comment 1.5: 2. Technical Clarity The AQT is a useful abstraction for organizing a transferred test case. In particular, the combination of normalized statement templates, read and write sets, and evolving schema context provides more structure than independent statement-level translation. The context ablation, where execution success decreases from 65.1% to 52.4%, provides evidence that cross-statement context improves target-side executability.

However, the precise role and scope of the explicit dependency graph remain unclear. Algorithm 1 defines an edge from statement (i) to statement (j) when $(i < j)$ and $(R_j \cap W_i \neq \varnothing)$. This formalization captures forward read-after-write dependencies. Because every edge follows the original order, the original statement sequence is already a valid topological ordering, and the stable topological sort in Algorithm 2 normally preserves that order.

It is therefore unclear what (G_{tc}) contributes independently beyond recording the dependencies and preserving an order that is already present. The reported ablation removes context propagation rather than the graph itself, so it establishes the value of context but not an independent benefit from (G_{tc}). The paper should explain whether the graph affects retrieval, prompting, repair, or any other step not apparent from the algorithms.

The paper should also define the content and construction of the "object-state summaries" in (D) and clarify which forms of state are represented. The current formal definition explicitly supports forward RAW dependencies and cumulative schema/type context. Broader descriptions such as preserving all schema, data, and state dependencies should be narrowed unless additional dependency forms are implemented.

These issues do not invalidate the sequence-aware design, but clarifying them would make the technical contribution more precise.
Reply: TODO.

Comment 1.6: 3. Semantic Correctness The paper explicitly states that its evaluation targets fuzzing-seed quality rather than source-to-target result equivalence. This is a reasonable design choice. For fuzzing, a transferred seed primarily needs to be executable on the target DBMS, retain useful SQL structures, and be compatible with downstream mutation. Full semantic equivalence is not necessarily required for this purpose.

However, the metric named "Semantic Correctness" only checks whether a seed executes on the target DBMS without a runtime error. This establishes execution validity rather than semantic equivalence. The distinction is important because repair operations such as constant substitution and minimal commenting can change test behavior.

I suggest renaming this metric to "Execution Validity" or "Runtime Validity" and narrowing statements concerning equivalent syntax, semantic-intent preservation, and high-fidelity transfer. A sampled source-to-target behavioral comparison would provide useful supplementary evidence.
Reply: TODO.

Comment 1.7: 4. Evaluation The evaluation provides strong evidence that the complete MORPH pipeline produces practically useful seeds. In particular, the confirmed bugs and improvements over SEDAR demonstrate that the approach can exercise target DBMS code that remains difficult to reach with existing seeds.

My main concern is the configuration of the translation baselines. The original CrackSQL evaluation focuses on PostgreSQL, MySQL, and Oracle, while RISE evaluates PostgreSQL-to-MySQL and PostgreSQL-to-Oracle translation. MORPH applies these systems to MonetDB, Virtuoso, DuckDB, and ClickHouse. The paper should explain how each baseline was extended to these target dialects, including the dialect documents, knowledge bases, translation rules, LLMs, prompts, and manual effort used. It should also explain how a statement- or artifact-level translator was applied to a complete test sequence and what schema context, if any, was provided.

The SEDAR reproduction also requires clarification. The original SEDAR mutability-refinement algorithm temporarily hides fuzzer-parser-incompatible regions and restores them after mutation. The paper currently summarizes this process as comment-out sanitization and attributes structural simplification to it. The authors should confirm whether their reproduction includes the restoration step. If the motivating example is an observed failure in the reproduced system rather than the general SEDAR algorithm, this distinction should be stated explicitly.

The paper reports average coverage over five runs, but Tables III and VII provide no variance, confidence intervals, or statistical tests. Such information is important for stochastic fuzzing experiments. It is also unclear whether the reported bug counts are unions across the five runs or results from a particular run.

The full bug-finding campaign discovers 28 bugs, whereas the 24-hour comparison reports 22. The duration and computational budget of the full campaign should be reported. Similarly, Table VIII should specify the currency of the cost, the unit of time, and the total offline cost of processing all test files.

Finally, the component ablation is performed only for PostgreSQL-to-DuckDB transfer. Extending at least the most important ablation, particularly context propagation, to additional source-target pairs would strengthen the generality of the findings.
Reply: TODO.

Comment 1.8: 6. Verifiability and transparency The paper reports the target DBMS versions, corpus size, primary model, temperature, hardware environment, and an anonymous artifact.

For full reproducibility, the paper should additionally document the baseline adaptations, source-target test allocation, repeated-run aggregation procedure, full campaign duration, and validation procedure for automatically generated translation rules. Table IX reports that 136 rules

required correction.
Reply: TODO.

Replies to Reviewer B

Comment 2.1: Could the authors clarify whether the reported coverage/bug-finding improvements primarily stem from AQT-based translation or from a larger/richer effective seed corpus? In a controlled experiment where all baselines use exactly the same number and source of initial seeds, how much of the 14% coverage gain over SEDAR remains attributable to AQT?
Reply: TODO.

Comment 2.2: Regarding the characterization of SEDAR's comment-and-restore mechanism—if both MORPH and SEDAR ultimately feed compatible seeds to GRIFFIN, what is the exact difference in their post-mutation handling? Do seeds repaired by MORPH produce semantically equivalent mutated queries, whereas SEDAR's approach loses that equivalence? If not, what is the claimed advantage?
Reply: TODO.

Comment 2.3: 1. Limited novelty. The paper positions AQT as a key contribution, but AQT largely resembles an existing SQL AST or intermediate representation augmented with abstract labels, read/write sets, dependency edges, and schema context. The authors do not clearly articulate what fundamentally new capability AQT provides over existing SQL representations (e.g., JOOQ's parse tree or SQLGlot's AST) or translation systems (CrackSQL, RISE, QTRAN). The distinction from these systems is insufficiently developed.

More importantly, the evaluation lacks compositional baselines such as "existing SQL translator + SEDAR/GRIFFIN" or "AST-based translation + GRIFFIN". Without such ablations, it is difficult to determine whether the contribution is a fundamentally new transfer technique or mainly an engineering integration of existing components. The authors should add such baselines or provide a clearer theoretical argument for why AQT enables capabilities that prior representations cannot achieve.
Reply: TODO.

Comment 2.4: 2. Incomplete discussion of closely related work. SmartFuzz (OOPSLA 2026) also studies high-quality seed generation for DBMS fuzzing and is directly relevant to the problem addressed by this paper. However, the paper neither discusses it sufficiently nor compares against it experimentally. The authors should add a discussion of SmartFuzz in Section VI (Related Work), clarify the differences in approach, and ideally include it as a baseline in the evaluation if feasible.
Reply: TODO.

Comment 2.5: 3. Potentially inaccurate characterization of SEDAR. The paper characterizes SEDAR's comment-and-restore mechanism as "destructive sanitization" that "sacrifices semantic depth." However, SEDAR's mechanism is designed to temporarily hide target-specific constructs from downstream fuzzers' strict parsers during mutation and restore them afterward. This is a pragmatic compatibility strategy, not a destructive removal of query logic.

The authors should revise their characterization to accurately reflect SEDAR's design intent and clarify the actual difference: MORPH performs pre-migration repair to make seeds parser-compatible, whereas SEDAR defers some compatibility handling until after mutation. A fairer comparison would evaluate both approaches under the same downstream fuzzer (e.g., GRIFFIN) and measure the net effect of each strategy on mutation effectiveness.
Reply: TODO.

Comment 2.6: Questionable classification in Table I.
Reply: We have clarified the criteria for the three symbols in Table I and revised the entries accordingly. SQLGlot's *Test Case Migration*, jOOQ's *Cross-stmt. Deps.*, and CrackSQL's *DML* entries are now marked as △. These revisions acknowledge SQLGlot's multi-statement transpilation and jOOQ's schema metadata tracking, while qualifying CrackSQL's DML support. Related corrections to the characterization of SEDAR are described in the reply to **Requested Change** 1.

Comment 2.7: 5. Unclear source of the improvement. The reported coverage and bug-finding improvements may stem from a larger or more diverse effective seed corpus rather than the AQT-based translation technique itself. For a fair comparison with SEDAR, GRIFFIN, and SQUIRREL, all approaches should use exactly the same source seed corpus and produce comparable numbers of final seeds that pass the target parser.

The current experimental setup does not control for seed corpus size. The authors should: (a) report the number of valid seeds generated by each method before fuzzing, (b) normalize the corpus size across baselines, or (c) conduct an experiment where seed counts are equalized. This controlled setting is necessary to isolate the contribution of MORPH's SQL translation technique and demonstrate that improvements are not simply due to differences in seed quantity or diversity.
Reply: TODO.

Comment 2.8: 6. Translation overhead is unclear. Table VIII reports "Cost/TS" and "Time/TS" but does not specify the unit of measurement (e.g., USD per test suite? seconds per test suite?) or the size of the test suite being translated. Additionally, the paper does not compare MORPH's end-to-end translation time and LLM cost with previous methods such as SEDAR.

The authors should: (a) clearly define the units for cost and time, (b) report the average number of statements per test suite, (c) report per-seed translation time and cost, and (d) compare these overhead metrics against SEDAR and other baselines to demonstrate the practical feasibility of MORPH in real-world deployment.
Reply: TODO.

Comment 2.9: Bug evidence is insufficient. The bug links are inaccessible, and the triggering test cases are not publicly available. The artifact includes complete and executable source code.
Reply: TODO.

Replies to Reviewer C

Comment 3.1: The "semantic correctness" metric means only "executes without runtime errors," not result equivalence; a seed that runs but computes something different still counts. Using "+130-450%" as a headline overstates what was measured.
Reply: TODO.

Comment 3.2: RQ2 baselines are somewhat hard to believe: zero bugs in 24 hours for SQUIRREL, GRIFFIN, and SQLancer across all four DBMSs. The baseline configuration is undocumented, several cells are "-", for unsupported targets, and the "528% over SQLancer" headline is computed over only the two DBMSs that SQLancer supports.
Reply: TODO.

Comment 3.3: Numeric inconsistency: the text claims RISE transfer improved semantic correctness "from 0.314 to 0.776," but Table V's own row reads RISE 0.314 / MORPH 0.748. One of these is wrong.
Reply: TODO.

Comment 3.4: Missing related work: SQLRight (USENIX Sec 2022) absent from RQ2; sqllogictest is never discussed; classic seed-selection literature ignored.
Reply: TODO.

Comment 3.5: What's novel here is the conjunction of approaches: AST abstraction + RAG + parser-feedback repair, which are established ideas assembled around the AQT.
Reply: TODO.

Comment 3.6: RQ1 (28 bugs, unspecified "full campaign") vs RQ2 (22 bugs in 24h) relationship and campaign duration are unclear.
Reply: TODO.

Comment 3.7: I would recommed tightening the validity claims: add a small differential check on transferred seeds (do they return equivalent results on a shared dialect subset?) to back the "semantic correctness" framing, and lean on coverage/bug counts rather than feature-richness as the primary evidence. For RQ2, publish baseline command lines and seed corpora in the artifact; the all-zero bug rows need more transparency, and SQLRight belongs in the comparison.
Reply: TODO.